Experiment-01: Genetic Algorithm (Binary representation)

- Genetic Algorithm (GA) is an optimization and search technique based on the principles of natural selection and genetics.
- It is a subset of Evolutionary Computation that mimics the biological process of evolution to solve complex problems.
- Instead of working on a single solution, GA maintains a population of candidate solutions and iteratively evolves them to find the best one.

Source Code:

selection.py

```
import random
def Tournament_selection(population_fitness):
    selected=[]
    cnt=0
    while cnt!=6:
        index1=random.randint(0,len(population_fitness)-1)
        index2=random.randint(0,len(population_fitness)-1)
        # selecting based on fitness value higher
        if population_fitness[index1][1]>population_fitness[index2][1]:
            selected.append(population_fitness[index1])

        elif population_fitness[index1][1]<population_fitness[index2][1]:
            selected.append(population_fitness[index2])

        elif population_fitness[index1][1]!=0 or population_fitness[index2][1]!=0:
            i=random.choice([index1,index2])
            selected.append(population_fitness[i])
        else:
            continue
        cnt+=1
    return selected
```

crossover.py

```
def single_point_crossover(selected):
    n=len(selected)
    point=3
    selected=[i[0] for i in selected]
    pair=[]
    for i in range(int(n/2)):
        pair.append((i,n-i-1)) # 1-6, 2-5,3-4 pair making

    offspring=[]
    for i in pair:
```

```
        p1=selected[i[0]]
        p2=selected[i[1]]

        o1=p1[0:point]+p2[point:]
        o2=p2[0:point]+p1[point:]
        offspring.append((o1,o2))
    return offspring
```

mutation.py

```
import random
def bit_flipping_mutation(offspring1,offspring2):
    # print(offspring)
    bit=random.randint(-1,4)
    if bit>=0:
        b=offspring1
        b=list(b)
        if b[bit]=='1': b[bit]='0'
        else: b[bit]='1'
        offspring1="".join(b)

    bit=random.randint(-1,4)
    if bit>=0:
        b=offspring2
        b=list(b)
        if b[bit]=='1': b[bit]='0'
        else: b[bit]='1'
        offspring2="".join(b)
    return (offspring1,offspring2)
```

Item	Value	Weight
1	10	2
2	7	3
3	12	4
4	8	5
5	15	7

Genetic Algorithm Implementation

```
from selection import Tournament_selection
from crossover import single_point_crossover
from mutation import bit_flipping_mutation

n=5
chromosome_fitness_value=[]
max_capacity=12

knapsack=[]
with open("fitness_value.txt",mode="r") as file:
    file=file.readlines()
    for i in file:
        v,w=i.split(' ')
        knapsack.append((int(v),int(w)))

def fitness(c): # calculating fitness value
    value=0
    weight=0
    for i in range(n):
        if c[i]=='1':
            value+=knapsack[i][0]
            weight+=knapsack[i][1]
    if weight<=max_capacity:
        return value
    else:
        return 0

for i in range(0,2**n): # fitness value of all the instance
    c=bin(i)[2:]
    c=f"{i:0{n}b}"
    chromosome_fitness_value.append((c,fitness(c)))

print(chromosome_fitness_value)
```

```
[('00000', 0), ('00001', 15), ('00010', 8), ('00011', 23), ('00100', 12), ('00101', 27), ('00110', 20), ('00111', 0), ('01000', 7), ('01001', 22), ('01010', 15), ('01011', 0), ('01100', 19), ('01101', 0), ('01110', 27), ('01111', 0), ('10000', 10), ('10001', 25), ('10010', 18), ('10011', 0), ('10100', 22), ('10101', 0), ('10110', 30), ('10111', 0), ('11000', 17), ('11001', 32), ('11010', 25), ('11011', 0), ('11100', 29), ('11101', 0), ('11110', 0), ('11111', 0)]
```

```
selected=Tournament_selection(chromosome_fitness_value)
for i in selected:
    print(i)
```

```
('11000', 17)
('11001', 32)
('01010', 15)
('10010', 18)
('11010', 25)
('00110', 20)
```

```
offspring=single_point_crossover(selected)
for i in offspring:
    print(f"({i[0]},{fitness(i[0])})")
    print(f"({i[1]},{fitness(i[1])})")
```

```
(11010,25)
(00100,12)
(11010,25)
(11001,32)
(01010,15)
(10010,18)
```

```
for i in offspring:
    new_offspring=bit_flipping_mutation(i[0],i[1])
    print(f"({new_offspring[0]},{fitness(new_offspring[0])})")
    print(f"({new_offspring[1]},{fitness(new_offspring[1])})")
```

Output: (Final)

```
(11011,0)
(00000,0)
(10010,18)
(10001,25)
(01011,0)
(10110,30)
```

Conclusion:

In conclusion, Genetic Algorithms are powerful, adaptive search heuristics that excel in finding near-optimal solutions in vast, complex, and non-differentiable search spaces. By simulating the evolutionary pressure of natural selection through selection, crossover, and mutation, they effectively explore the solution space without requiring gradient information, making them highly robust against getting trapped in local peaks. Key Applications are Optimization of complex engineering designs, Training Artificial Neural Networks, Robotics, Automated scheduling and timetable, financial modeling and trading strategy generation, Code breaking and cryptography, Feature selection in Machine Learning.